

hermetic_email_run.sh — companion guide (§11.4.18)

Revision: 1 **Last modified:** 2026-07-02T00:00:00Z **Status:** H2.email for the hermetic WireGuard test harness (design) — the fourth §11.4.52 **promotion** of an operator-gated protocol test to autonomous (after the Chromecast eureka leg, the FTP leg, and the WebDAV leg). Authority: inherits constitution/Constitution.md per §11.4.35.

Overview

tests/vpn_lan/hermetic_email_run.sh promotes the operator-gated tests/vpn_lan/email_roundtrip.sh to run **autonomously** (§11.4.52). That test runs four scored checks against a VPN-internal mail server:

- **T4.1 IMAPS (993)** — LOGIN + LIST "" "*" → PASS on a real ^* LIST line.
- **T4.2 SMTP submission (465 implicit-TLS)** — EHLO → AUTH LOGIN → MAIL → RCPT → DATA (body carries token=<tok>) → PASS on a **2xx** end-of-DATA code (sets the round-trip token).
- **T4.3 POP3S (995)** — USER/PASS/STAT/RETR → PASS **iff** the retrieved body contains the T4.2 token (a real send→retrieve round-trip; missing ⇒ **FAIL**, not SKIP).
- **T4.4 open-relay negative (§4.3)** — an **unauthenticated** relay to an external domain → PASS **iff** MAIL/RCPT is refused (4xx/5xx); a 2xx acceptance of the external RCPT is a **FAIL** (helix_proxy must never be an open relay).
- **T4.5 reverse leg** — documented client→server-only N/A, always an honest SKIP.

Because email connects **directly** (openssl s_client to host:port), the hermetic promotion needs **no forward proxy** (unlike the WebDAV leg). Inside one unshare -Urnm it: (1) stands up the H0-full kernel-WireGuard tunnel between two unprivileged netns (veth 10.9.0.x underlay + real WG overlay 10.10.0.x); (2) runs ONE pure-python3-stdlib TLS mail peer in netns B bound to the WG-only overlay 10.10.0.2 — implicit-TLS **SMTP submission**, **POP3S**, and **IMAPS** sharing one in-memory mailbox (ssl.SSLContext(PROTOCOL_TLS_SERVER) + load_cert_chain wrapping each listener; hand-rolled line handlers — smtpd/asyncore were removed in py3.12); (3) points the bridge contract + HELIX_MAIL_*

at that peer; (4) runs the **UNMODIFIED** `email_roundtrip.sh` inside netns A, where its `bridge_require` gate flips `SKIP`→`UP` and its four legs produce **real PASSes** over the encrypted tunnel — no operator, no Mullvad, no live VPN. **Zero host installs** (§11.4.122): the peer is `stdlib` (`ssl` + `socket` + `threading`), the client is `openssl` + `base64`.

Privileged binds, with an honest fallback (§11.4.6)

`unshare -Urm` maps our `uid` → `root` inside the user namespace, so the peer holds `CAP_NET_BIND_SERVICE` in netns B and the RFC 8314 implicit-TLS ports **465/995/993** bind directly (verified: the runs below all report `SMTP=465 POP3S=995 IMAPS=993`). If a privileged bind ever fails at runtime the peer falls back to high ports (`14650/19950/19930`) and **publishes the actual bound ports** to a `ports` file the harness reads before setting `HELIX_MAIL_*` — it **detects, never guesses**.

How the bridge contract is pointed at the hermetic peer

var	hermetic value
<code>HELIX_SWORD_DIR</code>	a throwaway dir (non-empty, satisfies <code>bridge_load</code>)
<code>HELIX_BRIDGE_CONNECT</code> / <code>_DISCONNECT</code>	<code>true</code>
<code>HELIX_BRIDGE_HEALTH</code>	a real TCP connect to <code>10.10.0.2:465</code> over the tunnel
<code>HELIX_BRIDGE_SUBNET</code>	<code>10.10.0.0/24</code>
<code>HELIX_BRIDGE_HOST</code>	<code>10.10.0.2</code>
<code>HELIX_MAIL_HOST</code>	<code>10.10.0.2</code>
<code>HELIX_MAIL_IMAPS_PORT</code> / <code>_POP3S_PORT</code> / <code>_SUBMISSION_PORT</code>	the actual bound ports (<code>993/995/465</code> , or the high-port fallbacks)
<code>HELIX_MAIL_SUBMISSION_TLS</code>	<code>implicit</code> (RFC 8314)
<code>HELIX_MAIL_RELAY_PROBE_PORT</code> / <code>_RELAY_PROBE_TLS</code>	the submission port / <code>implicit</code>
<code>HELIX_MAIL_EXTERNAL_RELAY_DOMAIN</code>	<code>example.com</code>
<code>HELIX_MAIL_USER</code>	<code>helix</code>
<code>HELIX_MAIL_PASS</code>	a fresh per-run random (never logged, §11.4.10)

The peer binds ONLY the overlay 10.10.0.2 (never the underlay 10.9.0.2 / 0.0.0.0), so reaching it from netns A is possible only through wg0 — a successful TLS dialog is itself proof of tunnel traversal (§11.4.111). HELIX_MAIL_FROM/_TO are deliberately **unset** — the promoted test defaults them to HELIX_MAIL_USER. HELIX_BRIDGE_MODE=hermetic documents intent; the promotion is realised through the contract vars.

Usage

```
tests/vpn_lan/hermetic_email_run.sh # PASS /  
SKIP / FAIL  
MAIL_MUT=openrelay tests/vpn_lan/hermetic_email_run.sh # §1.1  
golden-bad (T4.4)  
MAIL_MUT=droptoken tests/vpn_lan/hermetic_email_run.sh # §1.1  
golden-bad (T4.3)
```

Evidence: qa-results/vpn_lan/hermetic_email/<UTC-ts>_<pass|
mut_*>_<pid>/run.evidence (+ selffetch.smtp.txt /
selffetch.pop3.txt / selffetch.relay.txt and the promoted test's
captured email_roundtrip.stdout; the promoted test also writes its
own qa-results/vpn_lan/phase4/.../).

Anti-bluff design

The normal PASS is emitted **only** when the promoted email_roundtrip.sh exits 0 **and** its stdout carries real ^PASS: lines for **all four** scored legs (imaps_login_list, smtp_submission_send, pop3s_retrieve_roundtrip, open_relay_refused) **with zero** ^FAIL: — a SKIP-only run can never satisfy that (the test's bridge-UP PASS is emitted to /dev/null, so ^PASS: lines are real scored legs only).

Two golden-bad teeth (§1.1 / §11.4.107(10)). Each makes the UNMODIFIED test **FAIL-closed** (`_trc != 0` **and** a targeted ^FAIL: line), and the harness PASSes in mutation mode **only** when it confirms that:

- MAIL_MUT=openrelay → the peer accepts the unauthenticated external RCPT (250) → the test's **T4.4** open-relay guard fires (FAIL: open_relay_refused ... OPEN RELAY – unauthenticated external RCPT accepted (code 250)), proving the §4.3 guard is genuinely exercised, not a rubber-stamp.
- MAIL_MUT=droptoken → the peer stores the message **without** the round-trip token → the test's **T4.3** POP3S retrieve fires (FAIL: pop3s_retrieve_roundtrip ... sent token '...' not retrieved via POP3S after retries), proving the real send→retrieve assertion is genuinely exercised.

Each tooth makes **only its own target leg** FAIL; the other three legs still PASS in mutation mode, so the FAIL is unambiguously attributable to the mutation.

Self-fetch cross-check (§11.4.107 not-stale). Before running the promoted test the harness proves the mutation state itself over the tunnel: normal → an authenticated SMTP submit (235) → POP3S RETR byte round-trip of **this run's fresh nonce** must succeed; droptoken → that same round-trip must be **broken** (nonce absent); openrelay → an unauthenticated external RCPT must be **accepted** (2xx). This ties the eventual verdict to a real encrypted round-trip / a live mutation of *our* data, decoupled from the downstream grep string and forbidding a stale/wrong peer. A **coupling-contract guard** greps the promoted test for all four scored-leg description tokens, so a future rename fails with a clear diagnostic here.

The TLS close_notify gotcha (unique to the email leg). The peer sends a TLS close_notify (tls.unwrap() **before** close()). The UNMODIFIED test's tcp_open reachability probe is `printf 'QUIT\n' | openssl s_client -quiet ... && return 0` (this host has **no nc**, so it falls back to openssl), so each leg's reachability hinges on openssl's **exit code**. A bare close() leaves openssl with unexpected eof while reading → exit 1 → the leg would silently **SKIP** instead of PASS. FTP/WebDAV drive curl, not openssl, so they never hit this — it is specific to the email leg. **Additionally**, because openssl s_client -quiet implies -ign_eof (it never closes on stdin EOF — the *server* must close first), the IMAPS handler closes cleanly on the probe's bare QUIT (IMAP has no bare QUIT — it uses tag LOGOUT); without that the probe would hang to timeout → exit ≠ 0 → the IMAPS leg would SKIP. Both were caught by the §11.4.50 evidence, root-caused per §11.4.102, and fixed in the peer.

Host-safety self-bounding. The peer runs under nsenter ... timeout -k 2 90 (inside netns B) so an outer-SIGKILL orphan self-terminates (§12); the outer timeout 80 reaps the whole unshare tree first. TLS accepts carry a 25 s socket timeout so a stalled handshake cannot wedge a thread.

Ambient-env determinism (§11.4.50). Before invoking the promoted test the harness unsets the HELIX_MAIL_* it does not set (_FROM/_TO/_TIMEOUT/_PROBE_TIMEOUT) plus the test overrides (EMAIL_ROUNDTRIP_EVIDENCE_DIR, SVORD_BRIDGE_LIB, HELIX_REPO_ROOT), so a leftover ambient-shell export cannot perturb the promoted test. This is a false-negative guard only — it can never manufacture a bluff PASS.

Captured evidence (verified 2026-07-02)

Normal run embedded the promoted test's own output — **all four scored legs PASS:**

```
tunnel up: wg handshake=1782970857; mail peer @10.10.0.2 (netns B)
SMTP=465 POP3S=995 IMAPS=993 ready
self-fetch OK: authenticated SMTP submit (235) -> POP3S RETR byte
round-trip of this run's fresh nonce over the tunnel (§11.4.107
not-stale)
INFO: sword bridge reports UP – running live email round-trips
PASS: imaps_login_list(10.10.0.2:993) [evidence: .../phase4/.../
t4_1_imaps_list.txt]
PASS: smtp_submission_send(10.10.0.2:465/implicit) [evidence: .../
phase4/.../t4_2_smtp_send.txt]
PASS: pop3s_retrieve_roundtrip(10.10.0.2:995) [evidence: .../
phase4/.../t4_3_pop3s_message.txt]
PASS: open_relay_refused(10.10.0.2:465->@example.com)
[evidence: .../phase4/.../t4_4_open_relay_probe.txt]
SKIP: email_reverse_leg(... N/A §11.4.6) [reason:
topology_unsupported]
RESULT: no FAIL (PASS/SKIP only)
HE_PASS: the UNMODIFIED email_roundtrip.sh ran AUTONOMOUSLY over
the hermetic WireGuard tunnel ...
```

3/3 deterministic (§11.4.50). Golden-bad teeth fired against the real test:

```
# MAIL_MUT=openrelay
FAIL: open_relay_refused(10.10.0.2:465->@example.com) [reason:
OPEN RELAY – unauthenticated external RCPT accepted (code
250; ...)]
# MAIL_MUT=droptoken
FAIL: pop3s_retrieve_roundtrip(10.10.0.2:995) [reason: sent token
'helixproxy-vpnlan-p4-...' not retrieved via POP3S after retries
(...)]
```

Captured evidence holds **server responses only** — a leak scan confirmed the client AUTH base64 blobs never appear (the 334 VXNlcm5hbWU6 / 334 UGFzc3dvcmQ6 lines are the server's RFC 4954 Username:/Password: challenges, not credentials).

Honest scope (§11.4.6 / §11.4.3)

Proves the **client-side email protocol logic** (IMAPS LOGIN+LIST, implicit-TLS SMTP submission with AUTH LOGIN, POP3S send→retrieve round-trip, and the open-relay refusal decision) autonomously over an encrypted tunnel against a

controlled pure-stdlib peer. It does **not** prove the real mail server on the real Mullvad topology (that stays the §11.4.3 operator-gated confirmation), and the peer's mailbox

- responses are controlled fixtures — it exercises the test's client-side dialog and verdict logic, not a full RFC-complete mail server.

Underlay-sniff differential — N/A for this harness (§11.4.107 / FINDINGS §7.1)

The sibling `hermetic_{bridge,ftp,webdav}_run.sh` harnesses carry an AF_PACKET **underlay-sniff non-leak differential** — during the round-trip they capture on the underlay `veth0` and assert the per-run plaintext marker is ABSENT there while WG ciphertext (0x04 to :51820) flows, with a `SNIFF_MUT=plain` golden-bad that emits the marker in cleartext so the "plaintext-absent" assertion is proven load-bearing.

This differential is **deliberately NOT added to the email harness** (§11.4.6 honest boundary): the mail peer is **implicit-TLS** — `openssl s_client` wraps the payload in TLS *before* WireGuard encapsulates it, so the round-trip token is never in cleartext on the underlay **even if WG were absent**. "Plaintext-absent on the underlay" is therefore tautologically true regardless of the tunnel, so a sniff here would be a §11.4-forbidden **vacuous/bluff test** (green for the TLS reason, not the WG reason) that a `SNIFF_MUT=plain` tooth could not rescue. Email's tunnel-gating is instead proven by the overlay-only bind + the **§11.4.111 wrong-destination negative** (a fetch to the underlay 10.9.0.2 MUST fail) + the `wg show rx/tx` handshake counters.

Prerequisites / SKIP

`bash`, `unshare+nsenter`, `iproute2 ip` (wireguard link type), `wg`, host wireguard kernel module, `python3` (stdlib only), `openssl`, `base64`, `curl`, `timeout`, plus `tests/vpn_lan/email_roundtrip.sh` and `tests/lib/sword_bridge.sh`. Any missing → honest SKIP: (§11.4.3). Process-headroom guard SKIPS on a starved host (§12). An unknown `MAIL_MUT` value SKIPS (closed set: `openrelay` | `droptoken`).

Security (§11.4.10)

WireGuard private keys **and** a throwaway self-signed cert/key: `mode-0600 mktemp` inside the namespace, used by path, removed on exit, never logged. The mail account password is a **fresh per-run random**, passed to the peer and the promoted test via the

environment (never argv, so never in ps), and NEVER logged; captured evidence holds server responses only. The account is throwaway — never a real credential.

Related

- [hermetic_ftp_run.md](#) — the sibling H2.x FTP promotion.
- [hermetic_webdav_run.md](#) — the sibling H2.x WebDAV promotion.
- [hermetic_bridge_run.md](#) — the H2 template (Chromecast eureka leg).
- [hermetic_wg_roundtrip.md](#) — the H0-full tunnel this reuses.
- [tests/vpn_lan/email_roundtrip.sh](#) — the promoted protocol test (UNMODIFIED).

Last verified

2026-07-02 (host: ALT Linux kernel 6.12, wireguard module, wireguard-tools 1.0.20210914, python3 3.13 stdlib, OpenSSL 3.5.4, nc absent → openssl reachability probe). Normal PASS (all four legs) + both golden-bad teeth + 3/3 determinism.